

DMXAPI

[返回DMXAPI](#) [接口状态监控](#) [运维日志](#) | [LangChain中国社区](#) [联系我们](#)

DMXAPI 推荐主流大模型&使用代码示例

👑👑👑 LLM小萌新，体验2个是青铜，体验6个是黄金，体验10个上王者！

[马上接入](#) [示例代码](#) [报错说明](#) [常见问题](#)

🎁 API聚合，太方便了

无需逐个注册：不用再一个个注册中国所有大模型厂商开发后台了，有一个DMXAPI账号全搞定。

模型全覆盖：包括国内外超过100个大模型。

轻量使用,社群陪伴：加入LangChain中国社区超过10000名开发者一起成长,90后、00后 LLM开发者都在这里。

我对国内大模型的一点体会：我是本站站长康康。因为一直在折腾数据预处理，会需要在不同的模型测试不同的 Prompt，有了一点点体会。我的整体感觉是：国内大模型能力差距很小，大尺寸模型的能力基本都可以平替 GPT 3.5 了，但和 GPT-4 差距还是挺明显的。做RAG知识库国内模型完全胜任，但要做一些合同分析等，还是需要提升。我手头经常是几个模型一起串起来用，因为有些 prompt 你会发现就是某个模型更容易理解。我的体会是：在你迭代 prompt 前，可以先多换几个模型试试，比在一个模型上死磕 Prompt 好。

2025年主流大模型推荐	模型全称	官方开发者平台
DeepSeek（中国）：专业推理模型	deepseek-r1 复制	开发者平台
DeepSeek（中国）：强大的基座模型	deepseek-v3 复制	开发者平台
腾讯混元：微信生态整合，T1推理模型	h	开发者平台
阿里巴巴（中国）：数学与编程领先	q	开发者平台
字节跳动：高并发场景优化	D	开发者平台
百度文心：中文搜索生态整合	ERNIE-4.0-8K 复制	开发者平台
智谱GLM：清华系最强大模型	glm-4 复制	开



2025年主流大模型推荐	模型全称	官方开发者平台
KIMI月之暗面：KIMI 背后的大模型	moonshot-v1-32k 复制	开发者平台
OpenAI：推荐主流大模型，图片分析调用量最大的模型	gpt-4o 复制	开发者平台
OpenAI：目前全球API调用量最大的模型	gpt-4o-mini 复制	开发者平台
Anthropic：伦理与创意写作突出	claude-3-7-sonnet-20250219 复制	开发者平台
Google DeepMind：全球最强多模态大模型，文本、视频、音频、图片	gemini-2.5-pro-exp-03-25 复制	开发者平台
马斯克xAI：强大的全能模型	grok-3-beta 复制	开发者平台
Meta：开源标杆免费小模型	meta-llama/Meta-Llama-3.1-8B-Instruct 复制	开发者平台

知识普及：大模型 API 接口的速度衡量标准：

RPM (Requests Per Minute)

每分钟可以处理的请求数量。

RPD (Requests Per Day)

每天可以处理的请求数量。

IPM (Images Per Minute)

每分钟可以处理的图像数量。。

TPM (Tokens Per Minute)

每分钟可以处理的令牌数量。

TPD (Tokens Per Day)

每天可以处理的令牌数量。

响应时间

阿步（中文人工客服）

我在，请说~

Type a message here...

1

https://dmxapi.cn/models.html#code-block

2/18

从请求应用端发送到大模型收到响应的时间。



Python 示例1-1： 普通Post文本对话 [复制代码](#)

```
# 这是一个 DMXAPI 调用 API 的 Python 例子
import requests
import json

# -----
#          3秒步接入 DMXAPI :   修改 Key 和 Base url (https://www.dmxapi.cn)
# -----
url = "https://www.dmxapi.cn/v1/chat/completions"    # 这里不要用 openai base url,

payload = json.dumps({
    "model": "gpt-4o-mini", # 这里是你需要访问的模型，改成上面你需要测试的模型名称就可
    "messages": [
        {
            "role": "system",
            "content": "You are a helpful assistant."
        },
        {
            "role": "user",
            "content": "周树人和鲁迅是兄弟吗？"
        }
    ]
})
headers = {
    'Accept': 'application/json',
    'Authorization': 'sk-*****', # 这里方
    'User-Agent': 'DMXAPI/1.0.0 (https://www.dmxapi.cn)',
    'Content-Type': 'application/json'
}

response = requests.request("POST", url, headers=headers, data=payload)
print(response.text)
```



阿步（中文人工客服）
我在，请说~

Type a message here...

Python 示例1-2: Openai官方库使用例子 [复制代码](#)

```
# -----
# 在 Openai官方库 中使用 DMXAPI KEY 的例子
# 需要先 pip install openai
# -----
from openai import OpenAI

client = OpenAI(
    api_key="sk-*****", # 替换成你的 DMXAPI KEY
    base_url="https://www.dmxapi.cn/v1", # 需要改成DMXAPI的中转 https://www.dmxapi.cn/v1
)

chat_completion = client.chat.completions.create(
    messages=[
        {
            "role": "user",
            "content": "周树人和鲁迅是兄弟吗？",
        }
    ],
    model="gpt-4o-mini", # 替换成你先想用的模型全称， 模型全称可以在DMXAPI 模型价
)

print(chat_completion)
```

Python 示例1-3: 流式输出 [复制代码](#)

```
import json
import requests

url = "https://www.dmxapi.cn/v1/chat/completions"

payload = {
    "model": "gpt-4o-mini", # 模型名称
    "stream": True, # 流式输出True开启
    "messages": [
```



阿步（中文人工客服）
我在，请说~

Type a message here...

1

```

{"role": "system", "content": "You are a helpful assistant."},
{"role": "user", "content": "周树人和鲁迅是兄弟吗? "},
],
}
headers = {
    "Accept": "application/json",
    "Authorization": "sk-*****",
    "User-Agent": "DMXAPI/1.0.0 (https://www.dmxapi.cn)",
    "Content-Type": "application/json",
}

response = requests.post(url, headers=headers, json=payload, stream=True)

buffer = ""
for chunk in response.iter_content(chunk_size=None):
    if chunk:
        buffer += chunk.decode("utf-8")
        while "\n" in buffer:
            line, buffer = buffer.split("\n", 1)
            if line.strip() == "":
                continue
            if line.startswith("data: "):
                data_line = line[len("data: ") :].strip()
                if data_line == "[DONE]":
                    break
            else:
                try:
                    data = json.loads(data_line)
                    delta = data["choices"][0]["delta"]
                    content = delta.get("content", "")
                    print(content, end="", flush=True)
                except json.JSONDecodeError:
                    # 如果JSON解析失败,
                    buffer = line
                    break

```

阿步 (中文人工客服)
我在, 请说~

Type a message here...

Python 示例2: Json固定格式输出 代码展示 复制代码

```

from pydantic import BaseModel
from openai import OpenAI
from dotenv import load_dotenv
import json
from textwrap import dedent

# 加载环境变量, 例如 API key 等配置信息
load_dotenv()

# 设置 OpenAI API 的工厂名称, 默认为 "openai"
factory = "openai"

# 初始化 OpenAI 客户端, 传入 API key 和 base URL
client = OpenAI(
    api_key="sk-*****", # 替换为你的 DMXAPI key
    base_url="https://www.dmxapi.cn/v1/" # 这里是DMXAPI的 base url, 注意这里需要
)

# 定义一个产品信息类, 用于解析 API 返回的数据
class ProductInfo(BaseModel):
    product_name: str # 产品名称, 字符串类型
    price: float # 价格, 浮点数类型
    description: str # 产品描述, 字符串类型

# 定义一个提示信息, 用于请求模型返回 JSON 格式的产品信息
product_prompt = '''根据给出的产品进行分析, 按json格式用中文回答,json format:product'''

# 获取产品信息的函数, 传入用户的问题
def get_product_info(question: str):
    # 使用 OpenAI 客户端进行聊天模型的请求
    completion = client.beta.chat.completions.create(
        model="gpt-4o-2024-08-06", # 指定模型
        messages=[
            {"role": "system", "content": dedent(product_prompt)}, # 发送系统消息
            {"role": "user", "content": question},
        ],
        response_format=ProductInfo, # 指定返回的数据格式为 ProductInfo
    )

    # 返回模型解析的第一个选项的消息结果

```

阿步 (中文人工客服)

我在, 请说~

Type a message here...

1

```

return completion.choices[0].message.parsed

# 初始化一个空的产品信息字典
product_inform = {}

# 定义将解析的结果转换为 JSON 的函数
def transform2JSON(parsed_result):
    # print(parsed_result) # 打印解析结果

    # 将解析的结果存储到字典中
    product_inform["product_name"] = parsed_result.product_name
    product_inform["price"] = parsed_result.price
    product_inform["description"] = parsed_result.description

    # 将字典转换为 JSON 字符串并返回, ensure_ascii=False 允许中文字符正常显示
    return json.dumps(product_inform, ensure_ascii=False, indent=4)

# 定义用户输入的问题, 即一个产品信息的描述
question = "75寸小米电视机"

# 调用函数获取产品信息
result = get_product_info(question)

# 将解析结果转换为 JSON 格式并打印
json_result = transform2JSON(result)

```

Python 示例3: embedding代码例子 复制代码

```

import json

import requests

# API endpoint
url = "https://www.dmxapi.cn/v1/embeddings" # <----- 仅限 cn人民币站使

# API key (假设需要认证)
api_key = "sk-*****" # <----- 填入

```



阿步 (中文人工客服)
我在, 请说~

Type a message here...

1

```

# Headers
headers = {"Authorization": f"Bearer {api_key}", "Content-Type": "application/json"}

# Body parameters
data = {
    "input": "你好，把这句话变向量吧。",
    "model": "text-embedding-ada-002", # 假设使用的模型
}

# Make the POST request
response = requests.post(url, headers=headers, data=json.dumps(data))

# Check the response
if response.status_code == 200:
    embedding = response.json()
    print("Embedding created successfully:")
    print(embedding)
else:
    print(f"Failed to create embedding. Status code: {response.status_code}")

```



文档

Python 示例4： 2024-09-12 o1-preview 模型代码例子 复制代码

这是一个 DMXAPI 调用 API 的 Python 例子

```
import requests
```

```
import json
```

```
# -----
# 这里不要用 openai base url, 需要改成DMXAPI
```

```
# -----
url = "https://www.dmxapi.cn/v1/chat/completions"
```

```
payload = json.dumps({
```

```
    "model": "o1-mini-2024-09-12", # 01系列模型包括: o1-preview、o1-preview-2024-0
```

```
    "messages": [
```

```
        # ===== 下面 2个参数, 在使用 o1-preview 系列模型时, 不需要
```

```
        #{
```

阿步（中文人工客服）

我在，请说~

Type a message here...

1


```

# "role": "system",
# "content": "You are a helpful assistant."
#}, ===== 上面 2个参数, 在使用 o1-preview 系列模型时, 不需要填写 :
{
    "role": "user",
    "content": "周树人和鲁迅是兄弟吗?"
}
]
}))
headers = {
    'Accept': 'application/json',
    'Authorization': 'sk-*****', # 这里放
    'User-Agent': 'DMXAPI/1.0.0 (https://www.dmxapi.cn)', # 这里也改成 DMXAPI 的中
    'Content-Type': 'application/json'
}

response = requests.request("POST", url, headers=headers, data=payload, timeout=(1

```



文档

Python 示例5: LangChain使用例子 (测试成功时间: 2024-11-25)

复制代码

```

# -----
# 在 LangChain 中使用 DMXAPI KEY 的例子
# -----
from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-4o-mini",
    base_url="https://www.dmxapi.cn/v1",
    api_key="sk-*****"
)
text = "周树人和鲁迅是兄弟吗?"
print(model(text))

```

阿步 (中文人工客服)
我在, 请说~

Type a message here...

复制代码

[illegible]

```
import http.client
import json

# 定义 API 密钥和基本 URL
API_KEY = "sk-*****" # 请替换为你们自己的 API 密钥
API_HOST = "www.dmxapi.cn" # API 主机地址
API_ENDPOINT = "/v1/images/generations" # API 请求路径

# 请求参数
prompt_text = "科技感的店铺门口，店铺名称是DMXAPI" # 描述生成图像的提示词
model_name = "seedream-3.0" # 可选: dall-e-3, seedream-3.0, flux-schnell, flux-dev, ...
image_size = "1024x1024" # 图像尺寸 参考值: 1792x1024, 1024 x 1792, 1024x1024

# 构建请求的 JSON 数据
payload = json.dumps(
    {
        "prompt": prompt_text,
        "n": 1, # 生产图片数量，修改会报错，默认1就可以。
        "model": model_name,
        "size": image_size,
    }
)

# 定义请求头信息
headers = {
    "Authorization": f"Bearer {API_KEY}",
    "Accept": "application/json",
    "User-Agent": "DMXAPI/1.0.0 (https://www.dmxapi.cn)",
    "Content-Type": "application/json",
}

# 建立 HTTPS 连接
```

```

conn = http.client.HTTPSConnection(API_HOST)

# 发送 POST 请求
conn.request("POST", API_ENDPOINT, payload, headers)

# 获取响应并读取数据
res = conn.getresponse()
data = res.read()

# 输出结果

```

Python 示例7-1： gpt、claude、gemini 多模态网络图片解析示例

复制代码

```

import requests

# 将 URL 和 API key 提取为独立变量，便于管理和修改
域名 = "https://www.dmxapi.cn/" # 定义API的基础域名
API_URL = 域名 + "v1/chat/completions" # 完整的API请求URL
API_KEY = "sk-*****" ## <-----

# 图片URL（使用代码B中的图片URL）
IMAGE_URL = "https://pic.rmb.bdstatic.com/bjh/down/9bfc5b8f8f725467a1e3ccc4d7adc1

def analyze_image(api_url, api_key, image_url, prompt):
    """
    使用指定的API对图片进行分析。

    :param api_url: API端点URL
    :param api_key: API密钥
    :param image_url: 需要分析的图片URL
    :param prompt: 分析提示词
    :return: 分析结果文本
    """

    # 构建请求数据payload，包括所需的模型和消息内容
    payload = {

```



阿步（中文人工客服）
我在，请说~

Type a message here...

1

```

"model": "gemini-2.0-flash-thinking-exp-1219", # 指定使用的多模态AI模型
"messages": [
    {
        "role": "system",
        "content": [
            {"type": "text", "text": "你是一个图片分析助手。"}
        ], # 提供系统指令
    },
    {
        "role": "user",
        "content": [
            {
                "type": "image_url",
                "image_url": {
                    "url": image_url # 使用代码B中的图片URL
                },
            },
            {
                "type": "text",
                "text": prompt, # 使用用户指定的提示词
            },
        ],
    },
],
"temperature": 0.1, # 设置生成文本的随机性
"user": "DMXAPI", # 发送请求的用户标识
}

```

定义HTTP请求头，包括内容类型和身份验证信息

```

headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {api_key}",
    "User-Agent": "DMXAPI/1.0.0 (https://dmxapi.cn)"
}

```

try:

```

# 发送POST请求，将请求数据和头信息传入requests.post
response = requests.post(api_url, headers=headers, json=payload)

```

检查响应状态

阿步（中文人工客服）
我在，请说~

Type a message here...

```

if response.status_code == 200:
    result = response.json()
    # 提取并返回分析结果
    return result["choices"][0]["message"]["content"]
else:
    # 如果请求失败，打印错误信息
    print(f"请求失败，状态码: {response.status_code}")
    print(response.text)
    return None
except Exception as e:
    print(f"请求过程中发生异常: {e}")
    return None

if __name__ == "__main__":
    # 定义你的提示词
    prompt = "请详细描述这张图片的内容。"

    # 调用函数进行图片分析
    analysis_result = analyze_image(API_URL, API_KEY, IMAGE_URL, prompt)

    if analysis_result:
        print("图片分析结果:")
        print(analysis_result)

```

Python 示例7-2: gpt-4o 多模态本地图片解析示例

[复制代码](#)

```
import base64
```

```
import requests
```

```
def encode_image(image_path):
```

```
    """
```

```
    读取本地图片并编码为Base64字符串。
```

```
    """
```

```
    with open(image_path, "rb") as image_file:
```



阿步（中文人工客服）

我在，请说~

Type a message here...

1

```

        encoded_string = base64.b64encode(image_file.read()).decode("utf-8")
    return encoded_string

```

```

# 将 URL 和 API key 提取为独立变量，便于管理和修改
域名 = "https://www.dmxapi.cn/" # 定义API的基础域名
API_URL = 域名 + "v1/chat/completions" # 完整的API请求URL
API_KEY = "sk-*****" # <-----

# 本地图片路径
image_path = "33.png" # <----- 本地图片路
base64_image = encode_image(image_path) # 编码本地图片

# 创建请求数据payload，包括所需的模型和消息内容
payload = {
    "model": "chatgpt-4o-latest", # 指定使用的多模态AI模型，除了gpt-4o 也推荐使用 c
    "messages": [
        {
            "role": "system", # 系统角色信息，可以为空
            "content": "",
        },
        {
            "role": "user", # 用户角色的消息内容
            "content": [
                {"type": "text", "text": "请解释图片里说了哪些内容"}, # 发送文本消息
                {
                    "type": "image_url", # 发送图片URL
                    "image_url": {
                        # 使用Base64编码的本地图片
                        "url": f"data:image/png;base64,{base64_image}"
                    }
                },
            ],
        },
    ],
    "temperature": 0.1, # 设置生成文本的随机性
    "user": "DMXAPI", # 发送请求的用户标识
}

```

```

# 定义HTTP请求头，包括内容类型和身份验证信息

```



阿步（中文人工客服）

我在，请说~

Type a message here...

```

headers = {
    "Content-Type": "application/json", # 设置内容类型为JSON
    "Authorization": f"Bearer {API_KEY}", # 使用 f-string 动态插入 API_KEY, 进行身
    "User-Agent": f"DMXAPI/1.0.0 ({域名})", # 自定义的User-Agent, 用于识别客户端信,
}

# 发送POST请求, 将请求数据和头信息传入API, 获取响应
response = requests.post(API_URL, headers=headers, json=payload)

# 输出API的响应内容
print(response.text)

```

Python 示例8: gpt-4o 函数调用FunctionCall示例 [复制代码](#)

```

import http.client
import json

# 创建一个 HTTPS 连接对象, 连接到指定的域名
conn = http.client.HTTPSConnection("www.dmxapi.cn")

# 定义请求体, 包含了 GPT-4 模型的参数设置
payload = json.dumps(
    {
        "model": "gpt-4o", # <-----
        "max_tokens": 100,
        "temperature": 0.8,
        "stream": False, # 是否启用流式响应, 这里设置为 False, 表示不使用流式输出
        "messages": [{"role": "user", "content": "上海今天几度?"}],
        "tools": [ # 定义可用工具, 这里定义
            {
                "type": "function", # 工具类型为函数
                "function": {
                    "name": "get_current_weather",
                    "description": "获得天气信息", # 函数描述
                    "parameters": {
                        "type": "object", # 参数类型为对象
                        "properties": { # 参数属性

```



阿步 (中文人工客服)
我在, 请说~

Type a message here...

1

```

        "location": {
            "type": "string", # 地点参数的类型为字符串
            "description": "城市和州名, 例如: 上海, 中国", # 其
        },
        "unit": {
            "type": "string", # 温度单位类型为字符串
            "enum": [
                "celsius",
                "fahrenheit",
            ], # 支持的单位有摄氏度和华氏度
        },
    },
    "required": ["location"], # 必须提供地点参数
},
},
],
}
)

```

设置请求头部信息, 指定接受和发送的数据格式

```

headers = {
    "Accept": "application/json",
    "Authorization": "Bearer sk--*****",
    "Content-Type": "application/json",
}

```

发送 POST 请求到 API

```
conn.request("POST", "/v1/chat/completions", payload, headers)
```

获取服务器响应

```
res = conn.getresponse()
```

读取响应数据

```
data = res.read()
```

打印解码后的响应数据



阿步 (中文人工客服)

我在, 请说~

Type a message here...

Python 示例9: Openai whisper使用示例 [复制代码](#)

```
import json # 添加 json 库的导入
import requests

def q_voice_to_text(file_path_wav):
    url = "https://www.dmxapi.cn/v1/audio/transcriptions"

    payload = {"model": "whisper-1"}
    files = {"file": ("audio.mp3", open(file_path_wav, "rb"))}

    # 直接使用 API 密钥
    gpt_key = "sk-*****" ## <-----

    headers = {"Authorization": f"Bearer {gpt_key}"}

    # 发送请求
    response = requests.request("POST", url, headers=headers, data=payload, files=files)

    # 处理响应
    data = json.loads(response.text)

    # 获取返回的文本内容
    voice_text = data["text"] if data["text"] is not None else ""

    return voice_text

print(q_voice_to_text("C:\\kywpy\\jay.mp3")) ## <----- 这里改成你的
```

报错状态码说明

状态代码

400 Bad Request

请检查你的请求格式，一般都是客户端错误。例如 o1 系列模型不支持 system 参数。



阿步（中文人工客服）
我在，请说~

Type a message here...

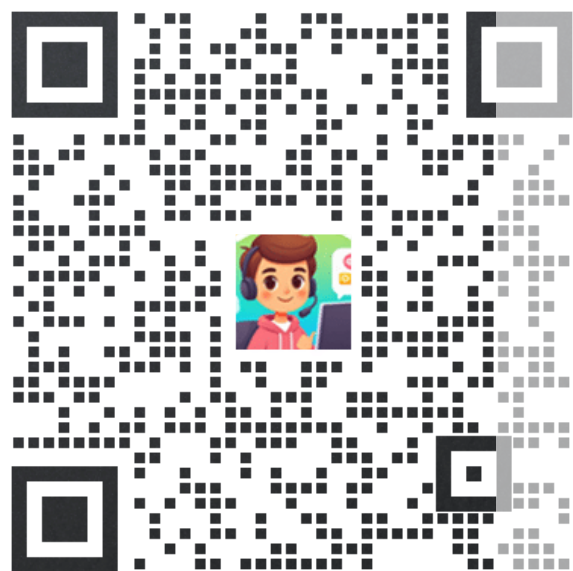
状态代码

代码解释

401 无效的令牌	API密钥验证未通过，换个模型测试来验证你的API密钥是否正确。如果你换模型正常，请联系管理员反馈处理。
403 令牌分组 XXX 已被禁用	一般是令牌权限问题或令牌限额用完。编辑令牌取消令牌限额 或 新建令牌可以解决。
404 Not Found	请检查 Base Url是否填写正确，尝试带上 /v1 或最后一个斜杠/。
413 Request Entity Too Large	prompt可能超长了。请缩短你的prompt，再次尝试，确认短prompt是否可以正常调用。
429 当前分组上游负载已饱和	openai对单个账号有限流，429说明后端某个账号并发太高遇到限流，请继续尝试。
500 Internal Server Error	服务器内部的错误。可能是中转服务器或OpenAI服务器的问题，不是你的问题。请再次尝试，多次持续报错请联系管理员处理。
503 当前分组NNN下对于模型 xxxx 无可用渠道	中转平台后端管理的问题，请联系管理员，让他帮你加上这个模型。请再次尝试，多次持续报错请联系管理员处理。
504 Gateway Timeout	504错误表示“网关超时”，未能在规定时间内从上游服务器获得响应。请再次尝试，多次持续报错请联系管理员处理。
524 连接超时	服务器没有在规定时间内完成请求，可能是DMXAPI通道拥挤造成。请再次尝试，多次持续报错请联系管理员处理。

 联系 管理员 & 客服

反馈问题 & 提建议 & [\[企业级服务咨询\]](#)，请扫我：



阿步（中文人工客服）
我在，请说~

Type a message here...